Types construits - Listes - Partie 1

1ere NSI

Fiche d'exercices sur la manipulation de listes, algorithmes de recherche d'occurences, compteur,

1 Fiche 1 — Les listes Python en 1 dimension

1ère NSI — durée indicative : 1 h

Rappels - Les indices commencent à **0** : le premier élément de `L` est `L[0]`, le dernier est `L[len(L)-1]`. - **Parcours par élément** : `for x in L:` ⟶ on lit les valeurs, sans connaître leur position. - **Parcours par indice** : `for i in range(len(L)):` ⟶ on connaît la position `i`, on peut donc **modifier** `L[i]`.

1.1 Partie 1 — Accès par indice *(≈ 15 min)*

Exercice 1 : Inventaire de jeu vidéo

Dans un jeu de rôle, le personnage possède un inventaire de 10 emplacements.

```
inventaire = ["épée", "potion", "bouclier", "vide", "clé", "vide", "arc", "carte",
"vide", "torche"]
```

Exemples

```
print(inventaire[2])        # Affiche : bouclier
inventaire[3] = "diamant"   # Modifie l'emplacement 3

# Échanger les éléments aux indices 1 et 5
temp = inventaire[1]
inventaire[1] = inventaire[5]
inventaire[5] = temp
```

Questions

1. Affichez l'objet situé à l'emplacement 4.
2. Le joueur utilise sa potion (indice 1) : remplacez-la par `"vide"`.
3. Écrivez une fonction `echange(L, i, j)` qui **modifie** la liste `L` en permutant les éléments d'indices `i` et `j`. Cette fonction ne renvoie rien.
4. Utilisez `echange` pour permuter l'épée (indice 0) et l'arc (indice 6).

Exercice 2 : Thermomètre intelligent

Un thermomètre enregistre la température de chaque heure de la journée (24 valeurs, de 0 h à 23 h).

```
temperatures = [12, 11, 10, 10, 9, 9, 11, 14, 17, 20, 23, 25, 27, 28, 27, 25, 22, 19,
17, 15, 14, 13, 13, 12]
```

Exemple

```
ecart = temperatures[10] - temperatures[3]
temperatures[5] += 3     # équivaut à temperatures[5] = temperatures[5] + 3
```

Questions

1. Affichez la température à 14 h.
2. Le capteur s'est trompé à 8 h : remplacez la valeur par 16.
3. Calculez l'écart entre la température de 13 h (maximum) et celle de 5 h (minimum).
4. Que se passe-t-il si on exécute `print(temperatures[24])` ? Pourquoi ? Quelle instruction affiche la **dernière** valeur sans connaître la longueur de la liste ?

1.2 Partie 2 — Parcourir une liste *(≈ 15 min)*

Exercice 3 : Le chevalet de Scrabble

```
chevalet = ['A', 'E', 'R', 'T', 'I', 'N', 'S']
```

Exemples

```
# Parcours par élément
for lettre in chevalet:
    print(lettre)

# Parcours par indice
for i in range(len(chevalet)):
    print("Position", i, ":", chevalet[i])

# Modifier en parcourant (par indice obligatoirement)
for i in range(len(chevalet)):
    if chevalet[i] == 'A':
        chevalet[i] = '*'

# Construire une nouvelle liste
voyelles_trouvees = []
for lettre in chevalet:
    if lettre in ['A', 'E', 'I', 'O', 'U', 'Y']:
        voyelles_trouvees.append(lettre)
```

Questions

1. Affichez « BINGO ! » si le chevalet contient au moins un `'S'`, sinon « Pas de S ».
2. Affichez chaque lettre avec sa position : `Position 0 : A`, `Position 1 : E`, etc.
3. Créez une nouvelle liste `consonnes` contenant uniquement les consonnes du chevalet.
4. Remplacez toutes les voyelles du chevalet par `'*'`. Pourquoi ne peut-on pas le faire avec `for lettre in chevalet` ?

1.3 Partie 3 — Compréhension de liste et mot-clé `in` *(≈ 10 min)*

Une **compréhension de liste** construit une liste en une seule ligne :

```
carres = [i**2 for i in range(4)]       # [0, 1, 4, 9]
pairs  = [x for x in range(10) if x % 2 == 0]   # [0, 2, 4, 6, 8]
```

Le mot-clé `in` teste l'appartenance et renvoie un booléen : `3 in [1, 2, 3]` vaut `True`.

Q1. Quelle instruction affecte la liste `[0, 1, 4, 9, 16]` à la variable `tableau` ?

- A. `tableau = [i**2 for i in range(4)]`
- B. `tableau = [i**2 for i in range(5)]`

- C. `tableau = [i**2 for i in range(16)]`
- D. `tableau = [i**2 for i in range(17)]`

Q2. Que vaut l'expression `[2 * k for k in range(5)]` ?

- A. `[0, 2, 4, 6, 8]`
- B. `[2, 4, 6, 8, 10]`
- C. `[0, 2, 4, 6, 8, 10]`
- D. `[2, 2, 2, 2, 2]`

Q3. Écrivez l'instruction qui teste si la liste `[0, 2, 4, 6, 8]` contient le nombre 4.

Q4. Que renvoie l'instruction `25 in [i**2 for i in range(16)]` ? Justifiez.

Q5. Réécrivez en une seule ligne (compréhension de liste) la construction de `voyelles_trouvees` de l'exercice 3.

1.4 Partie 4 — Algorithmes sur les listes *(≈ 20 min)*

Voici une fonction qui compte les **occurrences** d'un caractère dans un mot (ou une liste) :

```
def occurrences(mot, caractere):
    s = 0
    for c in mot:
        if c == caractere:
            s = s + 1
    return s
```

```
>>> occurrences(['A', 'E', 'R', 'T', 'I', 'N', 'S'], 'E')
1
>>> occurrences("banane", "a")
2
```

Exercice 4 : Compter

1. Adaptez `occurrences` pour écrire une fonction `nombre_de_voyelles(mot)` qui renvoie le nombre de voyelles d'un mot. Testez-la sur le chevalet.
2. Adaptez `occurrences` pour écrire une fonction `nombres_pairs(L)` qui renvoie le nombre de nombres pairs d'une liste d'entiers.

```
>>> nombres_pairs([3, 8, 12, 7, 0])
3
```

Exercice 5 : Rechercher

3. Complétez la fonction qui recherche un caractère précis dans une liste :

```
def recherche(L, caractere):
    trouve = False
    for c in L:
        if ..............:
            trouve = .........
    return .........
```

```
>>> recherche(['a', 'b', 'c', 'd'], 'b')
True
```

```
>>> recherche(['a', 'b', 'c', 'd'], 'e')
False
```

4. Adaptez ce script pour écrire la fonction `recherche_car_speciaux(L)`, qui renvoie `True` si la liste contient au moins un caractère parmi `@ & _ - ? / +`.

```
def recherche_car_speciaux(L):
    trouve = .........
    for c in L:
        if ..............:
            trouve = .........
    return .........
```

```
>>> recherche_car_speciaux(['@', 'b', 'c', 'd'])
True
>>> recherche_car_speciaux(['a', 'b', 'c', 'd'])
False
```

5. **Pour aller plus loin :** la fonction `recherche` continue de parcourir la liste même après avoir trouvé le caractère. Réécrivez-la avec un `return` placé dans la boucle pour qu'elle s'arrête dès que le caractère est trouvé.

Conseils : testez votre code au fur et à mesure, affichez des résultats intermédiaires avec `print()`, et vérifiez chaque fonction sur les exemples donnés.